In [27]:

```
import sys
import os
import importlib

# --------------------------
# Step 1: Clear ALL cached modules related to your project
# --------------------------
# List of modules to delete (covers models, src, data, metrics)
modules_to_clear = [
    m for m in sys.modules.keys()
    if any(p in m for p in ["models", "src", "data", "metrics"])
]
for module in modules_to_clear:
    del sys.modules[module]

# --------------------------
# Step 2: Set the EXACT project root (force Python to use this path)
# --------------------------
# Define the root of your whole_process folder (copy-paste from terminal)
PROJECT_ROOT = "/Users/darya/Desktop/Diploma/COMPAS - criminals/whole_pro

# Remove any existing paths that might conflict
sys.path = [p for p in sys.path if "COMPAS - criminals" not in p]

# Add the project root to the TOP of sys.path (highest priority)
sys.path.insert(0, PROJECT_ROOT)

# Verify the path (confirm it's first)
print("sys.path[0] =", sys.path[0])  # Should be your PROJECT_ROOT
cox_upd_path = os.path.join(PROJECT_ROOT, "models", "cox_model_upd.py")

# Load the file as a module (no package required)
spec = importlib.util.spec_from_file_location("cox_model_upd", cox_upd_pa
cox_model_upd = importlib.util.module_from_spec(spec)
spec.loader.exec_module(cox_model_upd)

# Extract the CoxModelUpd class (this is the one you need)
CoxModelUpd = cox_model_upd.CoxModelUpd

os.environ["NUMBA_DISABLE_JIT"] = "1"

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import importlib

from data.data_processor import DataProcessor
from data.splitter import SurvivalSplitter
from models.cox_model import CoxModel
from models.cox_model_upd import CoxModelUpd
from metrics.ibs import IBSMetric
from metrics.ibs_remain import IBSRemainMetric
from metrics.auprc import AUPRCMetric
from metrics.recurrent_count_error import RecurrentCountError
```

```
from pipeline.evaluator import SurvivalEvaluator
from survivors.constants import get_y
```

```
sys.path[0] = /Users/darya/Desktop/Diploma/COMPAS - criminals/whole_proces
s
```

In [28]:
```
df = pd.read_csv('data/individual_custody_timeline_rfm.csv')
print(df.columns)
```

```
Index(['name', 'age', 'sex', 'race', 'c_charge_degree', 'c_charge_desc',
       'in_custody', 'out_custody', 'custody_num', 'time_to_last_custody',
       'time_since_last_arrest', 'time_to_future_arrest', 'event',
       'event_of_future_arrest', 'curr_dur_custody', 'sum_dur_custody',
       'average_dur_custody', 'all_future_arrest_times', 'all_future_event
s'],
      dtype='object')
```

In [29]:
```
df
```

Out[29]:

| | name | age | sex | race | c_charge_degree | c_charge_desc | in_cu |
|---|---|---|---|---|---|---|---|
| **0** | aajah herrington | 23 | Female | African-American | (F3) | Driving While License Revoked | 201 |
| **1** | aajah herrington | 23 | Female | African-American | (F3) | Driving While License Revoked | 201 |
| **2** | aaliyah lovo | 21 | Female | Caucasian | (F2) | Burglary Unoccupied Dwelling | 2013- |
| **3** | aaron aprile | 27 | Male | Hispanic | (F3) | Grand Theft in the 3rd Degree | 201 |
| **4** | aaron aprile | 27 | Male | Hispanic | (F3) | Grand Theft in the 3rd Degree | 2014- |
| **...** | ... | ... | ... | ... | ... | ... | |
| **18663** | zolya isale | 21 | Female | African-American | NaN | NaN | 20 |
| **18664** | zuberi floyd | 22 | Male | African-American | (F3) | Uttering a Forged Instrument | 2013 |
| **18665** | zuberi floyd | 22 | Male | African-American | (F3) | Uttering a Forged Instrument | 20 |
| **18666** | zulimay lizcano | 44 | Female | Caucasian | (F3) | Grand Theft in the 3rd Degree | 20 |
| **18667** | zulma ramos | 40 | Female | Hispanic | (M2) | arrest case no charge | 201 |

18668 rows × 19 columns

In [30]:
```
# importlib.reload(data.data_processor)
```

```
processor = DataProcessor("data/individual_custody_timeline_rfm.csv")
cox_df = processor.load_and_prepare()
'''cox_df = pd.get_dummies(
    cox_df,
    columns=["sex", "race"],
    drop_first=True
)'''
cox_df.head()
```

Out[30]:

| | name | episode_col | entry | dur | event | age | time_since_last_arrest | avera |
|---|---|---|---|---|---|---|---|---|
| **0** | aajah herrington | 1 | 0.0 | 385.0 | 0 | 23 | 384.0 | |
| **1** | aaron aprile | 1 | 0.0 | 55.0 | 0 | 27 | 0.0 | |
| **2** | aaron davis | 1 | 0.0 | 432.0 | 0 | 41 | 314.0 | |
| **3** | aaron eddins | 1 | 0.0 | 104.0 | 0 | 37 | 104.0 | |
| **4** | aaron epstein | 1 | 0.0 | 297.0 | 1 | 22 | 296.0 | |

In [31]:

```
splitter = SurvivalSplitter(test_size=0.2)
train_df, test_df = splitter.split_by_individual(cox_df)
train_df = train_df.drop(columns=["curr_dur_custody"])
test_df = test_df.drop(columns=["curr_dur_custody"])

print("Train individuals:", train_df["name"].nunique())
print("Test individuals:", test_df["name"].nunique())
```

```
Train individuals: 3382
Test individuals: 845
```

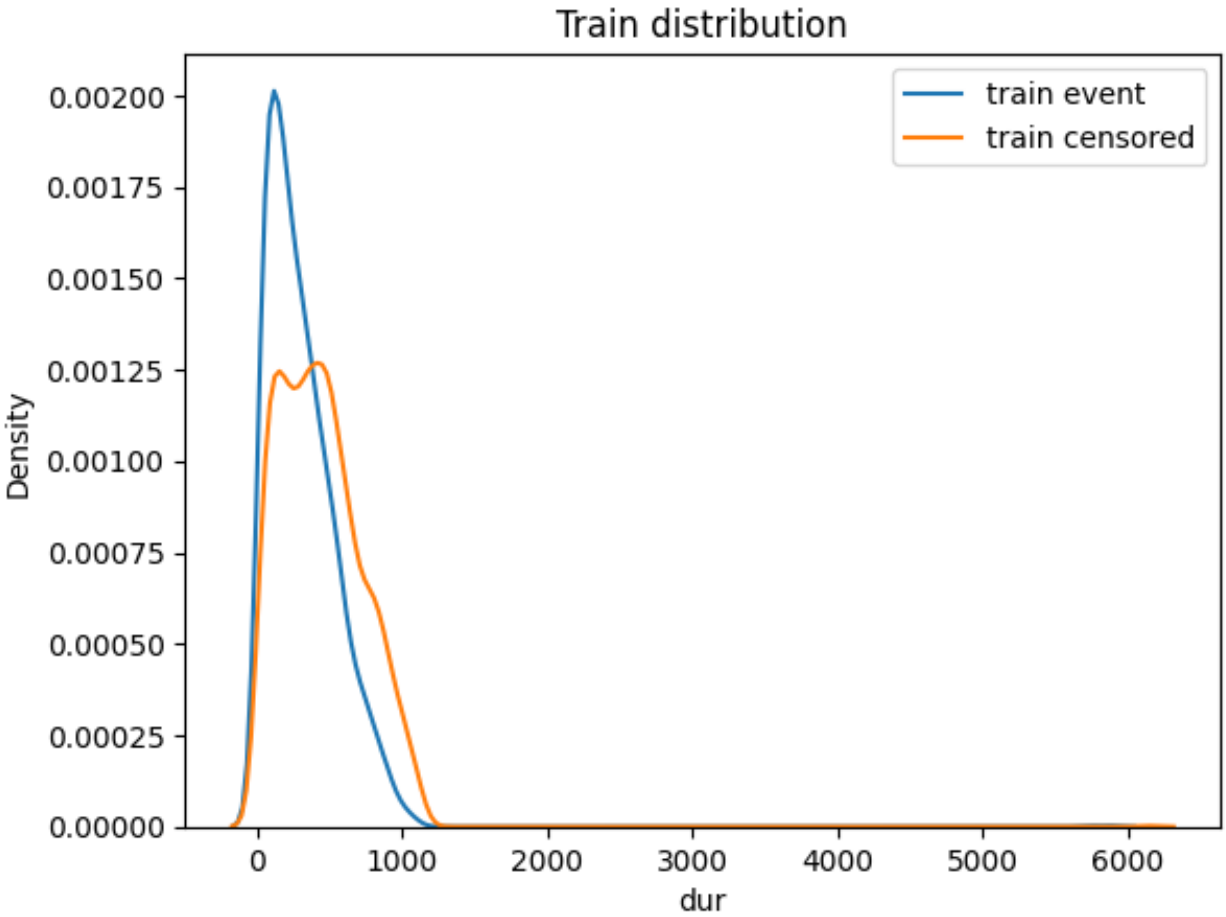
In [32]:

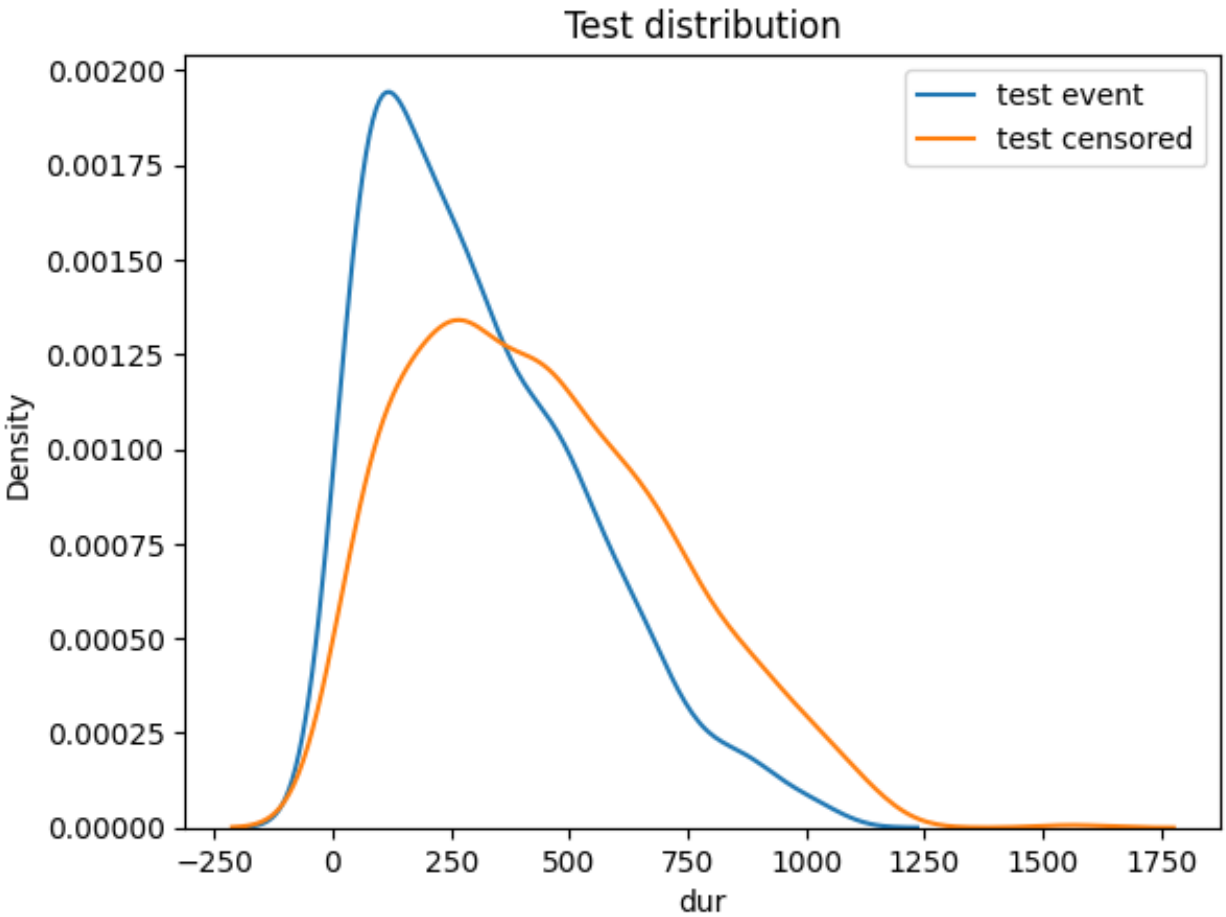
```
train_df["time"] = train_df["entry"] + train_df["dur"]
test_df["time"] = test_df["entry"] + test_df["dur"]

train_cens = train_df["event"].astype(bool)
test_cens = test_df["event"].astype(bool)

survival_train = get_y(
    cens=train_cens,
    time=train_df["time"],
    competing=False
)

survival_test = get_y(
    cens=test_cens,
    time=test_df["time"],
    competing=False
)
```

In [33]:

```
plt.figure()
sns.kdeplot(train_df[train_df["event"]==1]["dur"], label="train event")
```

```
sns.kdeplot(train_df[train_df["event"]==0]["dur"], label="train censored"
plt.legend()
plt.title("Train distribution")
plt.show()

plt.figure()
sns.kdeplot(test_df[test_df["event"]==1]["dur"], label="test event")
sns.kdeplot(test_df[test_df["event"]==0]["dur"], label="test censored")
plt.legend()
plt.title("Test distribution")
plt.show()

print("Train events:", train_df["event"].value_counts())
print("Test events:", test_df["event"].value_counts())
```






```
Train events: event
0    3382
1    2767
Name: count, dtype: int64
Test events: event
0    845
1    778
Name: count, dtype: int64
```

In [34]:

```python
# importlib.reload(models.cox_model)

print(train_df.columns)

features = [c for c in train_df.columns
            if c not in ["name", "entry", "time", "event", "dur", "episod
print(features)

cols_for_model = ["name", "entry", "time", "event"] + features
train_model_df = train_df[cols_for_model].copy()


model = CoxModel(features=features)
model.fit(train_model_df)
```

```
Index(['name', 'episode_col', 'entry', 'dur', 'event', 'age',
       'time_since_last_arrest', 'average_dur_custody', 'time'],
      dtype='object')
['age', 'time_since_last_arrest', 'average_dur_custody']
```

| | |
|---|---|
| **model** | lifelines.CoxPHFitter |
| **duration col** | 'time' |
| **event col** | 'event' |
| **entry col** | 'entry' |
| **cluster col** | 'name' |
| **robust variance** | True |
| **baseline estimation** | breslow |
| **number of observations** | 6149 |
| **number of events observed** | 2767 |
| **partial log-likelihood** | -20405.07 |
| **time fit was run** | 2026-03-03 13:14:52 UTC |

| | coef | exp(coef) | se(coef) | coef lower 95% | coef upper 95% | exp(coef) lower 95% | exp(coef upper 95% |
|---|---|---|---|---|---|---|---|
| **age** | -0.00 | 1.00 | 0.00 | -0.01 | -0.00 | 0.99 | 1.0 |
| **time_since_last_arrest** | -0.00 | 1.00 | 0.00 | -0.00 | -0.00 | 1.00 | 1.0 |
| **average_dur_custody** | -0.00 | 1.00 | 0.00 | -0.00 | -0.00 | 1.00 | 1.0 |

| | |
|---|---|
| **Concordance** | 0.74 |
| **Partial AIC** | 40816.15 |
| **log-likelihood ratio test** | 1166.16 on 3 df |
| **-log2(p) of ll-ratio test** | 836.44 |

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [35]:

```python
# RECURRENT WITH EPISODE_COL
features1 = [c for c in train_df.columns
            if c not in ["name", "entry", "time", "event", "dur"]]

cols_for_model1 = ["name", "episode_col", "entry", "time", "event"] + \
                 [c for c in features if c != "episode_col"]

train_model_df1 = train_df[cols_for_model1].copy()

model1 = CoxModelUpd(
    features=[c for c in features1 if c != "episode_col"],
    mode="recurrent",
    use_episode=True
)

model1.fit(train_model_df1)
```

| | |
|---|---|
| **model** | lifelines.CoxPHFitter |
| **duration col** | 'time' |
| **event col** | 'event' |
| **entry col** | 'entry' |
| **cluster col** | 'name' |
| **robust variance** | True |
| **strata** | episode_col |
| **baseline estimation** | breslow |
| **number of observations** | 6149 |
| **number of events observed** | 2767 |
| **partial log-likelihood** | -16050.54 |
| **time fit was run** | 2026-03-03 13:14:53 UTC |

| | coef | exp(coef) | se(coef) | coef lower 95% | coef upper 95% | exp(coef) lower 95% | exp(coef uppe 95% |
|---|---|---|---|---|---|---|---|
| **age** | -0.01 | 0.99 | 0.00 | -0.01 | -0.00 | 0.99 | 1.0 |
| **time_since_last_arrest** | -0.01 | 0.99 | 0.00 | -0.02 | -0.01 | 0.98 | 0.9 |
| **average_dur_custody** | -0.01 | 0.99 | 0.00 | -0.02 | -0.01 | 0.98 | 0.9 |

| | |
|---|---|
| **Concordance** | 0.95 |
| **Partial AIC** | 32107.08 |
| **log-likelihood ratio test** | 4470.49 on 3 df |
| **-log2(p) of ll-ratio test** | inf |

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [36]:

```python
# RECURRENT WITHOUT EPISODE_COL
features2 = [c for c in train_df.columns
             if c not in ["name", "entry", "time", "event", "dur", "episod

cols_for_model2 = ["name", "entry", "time", "event"] + features
train_model_df2 = train_df[cols_for_model2].copy()

model2 = CoxModelUpd(
    features=features2,
    mode="recurrent",
    use_episode=False
)
model2.fit(train_model_df2)
```

| | |
|---|---|
| **model** | lifelines.CoxPHFitter |
| **duration col** | 'time' |
| **event col** | 'event' |
| **entry col** | 'entry' |
| **cluster col** | 'name' |
| **robust variance** | True |
| **baseline estimation** | breslow |
| **number of observations** | 6149 |
| **number of events observed** | 2767 |
| **partial log-likelihood** | -20405.07 |
| **time fit was run** | 2026-03-03 13:14:54 UTC |

| | coef | exp(coef) | se(coef) | coef lower 95% | coef upper 95% | exp(coef) lower 95% | exp(coef upper 95% |
|---|---|---|---|---|---|---|---|
| **age** | -0.00 | 1.00 | 0.00 | -0.01 | -0.00 | 0.99 | 1.0 |
| **time_since_last_arrest** | -0.00 | 1.00 | 0.00 | -0.00 | -0.00 | 1.00 | 1.0 |
| **average_dur_custody** | -0.00 | 1.00 | 0.00 | -0.00 | -0.00 | 1.00 | 1.0 |

| | |
|---|---|
| **Concordance** | 0.74 |
| **Partial AIC** | 40816.15 |
| **log-likelihood ratio test** | 1166.16 on 3 df |
| **-log2(p) of ll-ratio test** | 836.44 |

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [37]:

```
# INDEPENDENT
train_ind = train_df.drop_duplicates("name").copy()

features3 = [c for c in train_ind.columns
             if c not in ["name", "entry", "time", "event", "dur", "episod

cols_for_model3 = ["time", "event"] + features3
train_model_df3 = train_ind[cols_for_model3].copy()

model3 = CoxModelUpd(
    features=features3,
    mode="independent"
)

model3.fit(train_model_df3)
```

```
/Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/site-pac
kages/lifelines/fitters/coxph_fitter.py:1614: ConvergenceWarning: Newton-R
aphson failed to converge sufficiently. Please see the following tips in t
he lifelines documentation: https://lifelines.readthedocs.io/en/latest/Exa
mples.html#problems-with-convergence-in-the-cox-proportional-hazard-model
  warnings.warn(
```

| | |
|---|---|
| **model** | lifelines.CoxPHFitter |
| **duration col** | 'time' |
| **event col** | 'event' |
| **baseline estimation** | breslow |
| **number of observations** | 3382 |
| **number of events observed** | 1503 |
| **partial log-likelihood** | -7574.77 |
| **time fit was run** | 2026-03-03 13:14:56 UTC |

| | coef | exp(coef) | se(coef) | coef lower 95% | coef upper 95% | exp(coef) lower 95% | exp(coef uppe 95% |
|---|---|---|---|---|---|---|---|
| **age** | -0.01 | 0.99 | 0.00 | -0.01 | -0.00 | 0.99 | 1.0 |
| **time_since_last_arrest** | -0.03 | 0.97 | 0.00 | -0.04 | -0.03 | 0.96 | 0.9 |
| **average_dur_custody** | -0.03 | 0.97 | 0.00 | -0.04 | -0.03 | 0.97 | 0.9 |

| | |
|---|---|
| **Concordance** | 1.00 |
| **Partial AIC** | 15155.53 |
| **log-likelihood ratio test** | 7152.59 on 3 df |
| **-log2(p) of ll-ratio test** | inf |

```
<class 'lifelines.fitters.coxph_fitter.CoxPHFitter'>
```

In [38]:
```
train_df["dur"].describe()
```

Out[38]:
```
count    6149.000000
mean      381.495853
std       289.923100
min         1.000000
25%       159.000000
50%       336.000000
75%       547.000000
max      6141.000000
Name: dur, dtype: float64
```

In [39]:
```
survival_test = get_y(
    cens=test_cens,
    time=test_df["time"],
    competing=False
```

```
)
```

In [40]:
```
test_df
```

Out[40]:

| | name | episode_col | entry | dur | event | age | time_since_last_arrest | av |
|---|---|---|---|---|---|---|---|---|
| **0** | aaron hammond | 1 | 0.0 | 699.0 | 0 | 25 | 688.0 | |
| **1** | aaron villacampa | 1 | 0.0 | 107.0 | 0 | 26 | 106.0 | |
| **2** | abraham ballestas | 1 | 0.0 | 35.0 | 0 | 52 | 34.0 | |
| **3** | acie canteen | 1 | 0.0 | 215.0 | 0 | 27 | 214.0 | |
| **4** | adam gillstrap | 1 | 0.0 | 630.0 | 0 | 35 | 628.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **1618** | zachary payton | 1 | 0.0 | 34.0 | 0 | 28 | 33.0 | |
| **1619** | zachary whalen | 1 | 0.0 | 634.0 | 0 | 24 | 632.0 | |
| **1620** | zaheer khan | 1 | 0.0 | 184.0 | 0 | 26 | 183.0 | |
| **1621** | zhivago walker | 1 | 0.0 | 126.0 | 0 | 30 | 125.0 | |
| **1622** | zuberi floyd | 1 | 0.0 | 757.0 | 0 | 22 | 756.0 | |

1623 rows × 9 columns

In [41]:
```
from lifelines.utils import concordance_index
#times = np.linspace(0, train_df["dur"].max(), 200)
horizon = np.quantile(train_df["time"], 0.95)
times = np.linspace(0, horizon, 200)

test_features = test_df[features].copy()
predictions = model.predict_survival(test_features, times)
#predictions = model.predict_survival(test_df, times)
estimate = predictions.values
estimate = np.asarray(estimate, dtype=np.float64)
times = np.asarray(times, dtype=np.float64)

survival_train = survival_train.astype([("event", "?"), ("time", "f8")])
survival_test = survival_test.astype([("event", "?"), ("time", "f8")])
print("Estimate shape:", estimate.shape)
print("Test size:", len(test_df))
print(predictions.shape)
print(test_df["name"].nunique())
print(len(test_df))
```

```
print()
print(type(estimate))
print(type(times))
print(estimate.dtype)
print(times.dtype)
print()
print(type(survival_train))
print(type(survival_test))
print(survival_train.dtype)
print(survival_test.dtype)

survival_train = survival_train.astype([("event", "?"), ("time", "f8")])
survival_test = survival_test.astype([("event", "?"), ("time", "f8")])

print(survival_test.dtype.names)
ibs_metric = IBSMetric()
mean_ibs, ibs_by_time = ibs_metric.compute(
    survival_train,
    survival_test,
    estimate,
    times
)

print(mean_ibs)

survival_train_cens = survival_train.astype([("cens", "?"), ("time", "f8"
survival_test_cens = survival_test.astype([("cens", "?"), ("time", "f8")]

auprc_metric = AUPRCMetric()
auprc = auprc_metric.compute(
    survival_train_cens,
    survival_test_cens,
    estimate,
    times
)
print("AUPRC:", np.mean(auprc))

ci_metric = concordance_index(
    test_df["dur"],
    np.trapz(predictions.values, times, axis=1),
    test_cens
)
print("Concordance Index:", ci_metric)
```

```
Estimate shape: (1623, 200)
Test size: 1623
(1623, 200)
845
1623

<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
float64
float64

<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
[('event', '?'), ('time', '<f8')]
[('event', '?'), ('time', '<f8')]
('event', 'time')
0.2054054924976589
AUPRC: 0.5666575296287015
Concordance Index: 0.7466311236238385
```

In [42]:
```
test_df
```

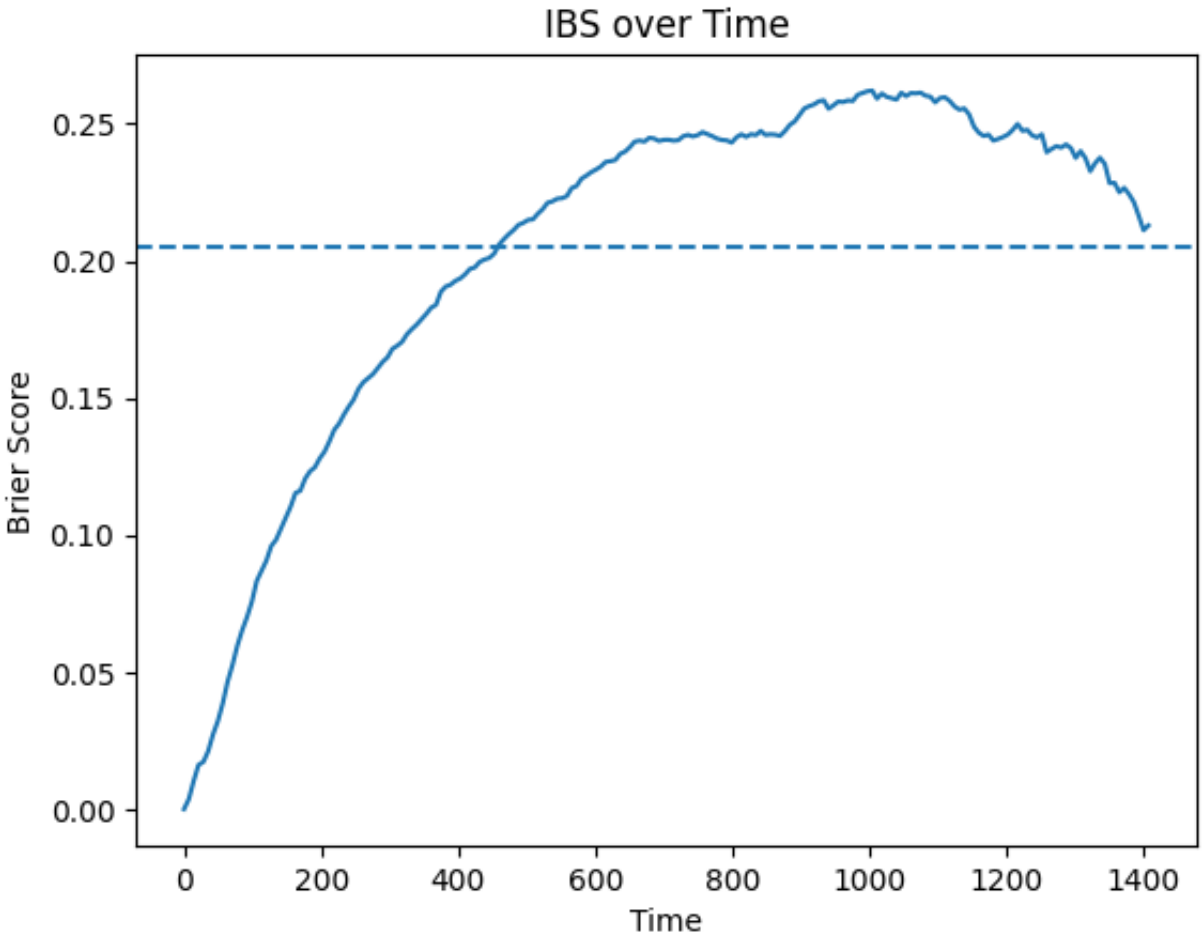
Out[42]:

| | name | episode_col | entry | dur | event | age | time_since_last_arrest | av |
|---|---|---|---|---|---|---|---|---|
| **0** | aaron hammond | 1 | 0.0 | 699.0 | 0 | 25 | 688.0 | |
| **1** | aaron villacampa | 1 | 0.0 | 107.0 | 0 | 26 | 106.0 | |
| **2** | abraham ballestas | 1 | 0.0 | 35.0 | 0 | 52 | 34.0 | |
| **3** | acie canteen | 1 | 0.0 | 215.0 | 0 | 27 | 214.0 | |
| **4** | adam gillstrap | 1 | 0.0 | 630.0 | 0 | 35 | 628.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **1618** | zachary payton | 1 | 0.0 | 34.0 | 0 | 28 | 33.0 | |
| **1619** | zachary whalen | 1 | 0.0 | 634.0 | 0 | 24 | 632.0 | |
| **1620** | zaheer khan | 1 | 0.0 | 184.0 | 0 | 26 | 183.0 | |
| **1621** | zhivago walker | 1 | 0.0 | 126.0 | 0 | 30 | 125.0 | |
| **1622** | zuberi floyd | 1 | 0.0 | 757.0 | 0 | 22 | 756.0 | |

1623 rows × 9 columns

In [43]:
```
plt.plot(times, ibs_by_time)
plt.axhline(y=mean_ibs, linestyle="--")
```

```
plt.title("IBS over Time")
plt.xlabel("Time")
plt.ylabel("Brier Score")
plt.show()
```




In [44]:
```
from survivors import metrics
test_df_results = test_df[["name", "episode_col", "dur", "event"]].copy()

test_df_results["dur"] = test_df["dur"] - test_df["entry"]

test_df_results["ibs"] = metrics.ibs(
            survival_train=survival_train,
            survival_test=survival_test,
            estimate=estimate,
            times=times,
            axis=0
        )
test_df_results["auprc"] = metrics.auprc(
            survival_train=survival_train,
            survival_test=survival_test,
            estimate=estimate,
            times=times,
            axis=0
        )
```

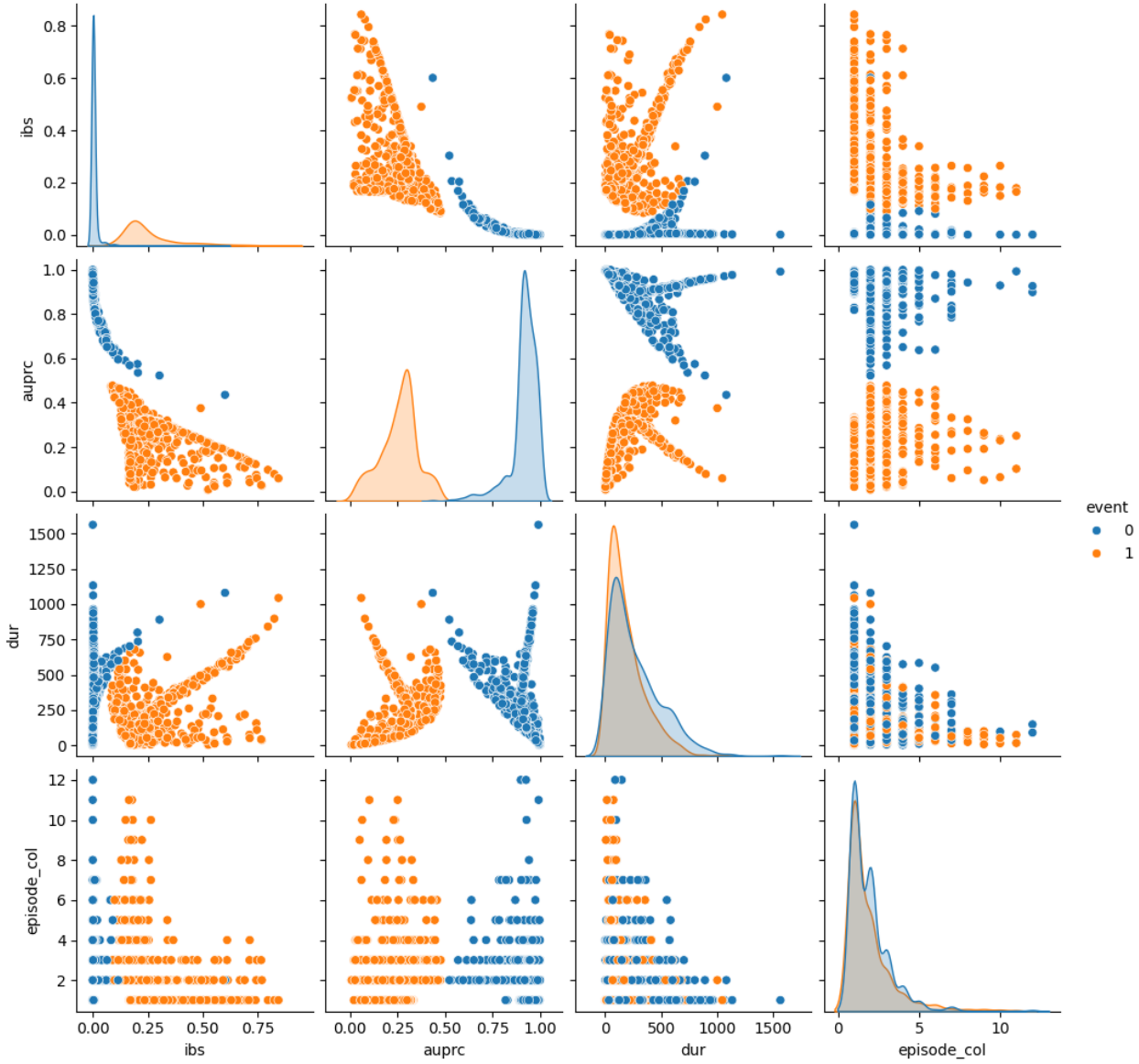
```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[44], line 13
      4 test_df_results["dur"] = test_df["dur"] - test_df["entry"]
      6 test_df_results["ibs"] = metrics.ibs(
      7             survival_train=survival_train,
      8             survival_test=survival_test,
   (...)
     11             axis=0
     12         )
---> 13 test_df_results["auprc"] = metrics.auprc(
     14             survival_train=survival_train,
     15             survival_test=survival_test,
     16             estimate=estimate,
     17             times=times,
     18             axis=0
     19         )

File /Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/site-packages/survivors/metrics.py:64, in auprc(survival_train, survival_test, estimate, times, axis)
     62 def auprc(survival_train, survival_test, estimate, times, axis=-1):
     63     time = survival_test["time"]
---> 64     event = survival_test["cens"]
     66     steps = np.linspace(1e-5, 1 - 1e-5, 100)
     67     before_time = np.dot(time[:, np.newaxis], steps[np.newaxis, :])

ValueError: no field of name cens
```

In [ ]:

```
# sns.scatterplot(x="episode_col", y="auprc", data=test_df_results)
sns.pairplot(test_df_results, vars=["ibs", "auprc", "dur", "episode_col"]
```

Out[ ]:

```
<seaborn.axisgrid.PairGrid at 0x17727ab60>
```




In [ ]:
```
test_df_results[test_df_results["episode_col"] >= 8].round(3)
# test_df["dur"].describe()
```

Out[ ]:

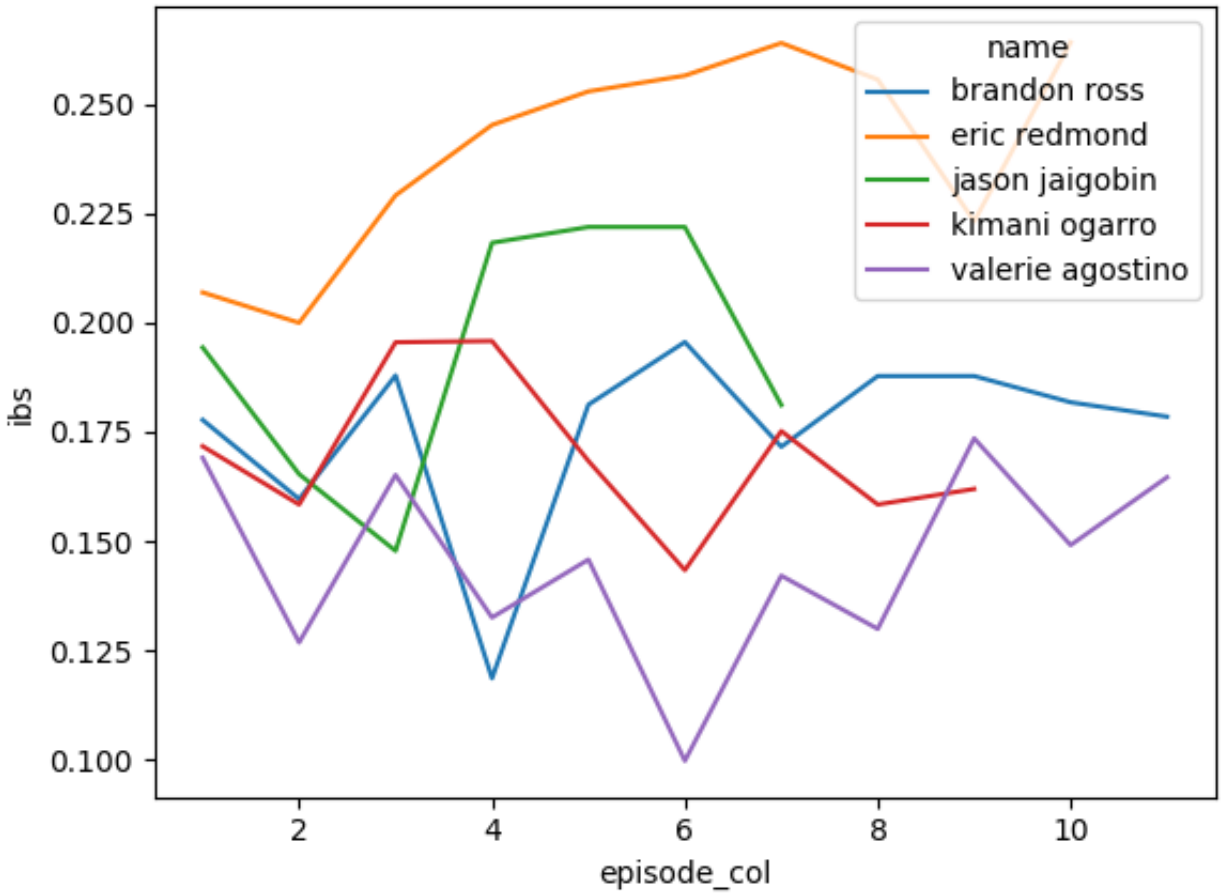
| | name | episode_col | dur | event | ibs | auprc |
|---|---|---|---|---|---|---|
| **216** | brandon ross | 8 | 47.0 | 1 | 0.188 | 0.192 |
| **217** | brandon ross | 9 | 47.0 | 1 | 0.188 | 0.192 |
| **218** | brandon ross | 10 | 67.0 | 1 | 0.182 | 0.237 |
| **219** | brandon ross | 11 | 74.0 | 1 | 0.178 | 0.251 |
| **220** | brandon ross | 12 | 148.0 | 0 | 0.001 | 0.898 |
| **592** | eric redmond | 8 | 23.0 | 1 | 0.256 | 0.095 |
| **593** | eric redmond | 9 | 100.0 | 1 | 0.223 | 0.253 |
| **594** | eric redmond | 10 | 14.0 | 1 | 0.264 | 0.065 |
| **595** | eric redmond | 11 | 18.0 | 0 | 0.000 | 0.992 |
| **770** | jason jaigobin | 8 | 99.0 | 0 | 0.000 | 0.942 |
| **1011** | kimani ogarro | 8 | 79.0 | 1 | 0.158 | 0.274 |
| **1012** | kimani ogarro | 9 | 74.0 | 1 | 0.162 | 0.264 |
| **1013** | kimani ogarro | 10 | 97.0 | 0 | 0.000 | 0.929 |
| **1554** | valerie agostino | 8 | 97.0 | 1 | 0.130 | 0.325 |
| **1555** | valerie agostino | 9 | 7.0 | 1 | 0.174 | 0.052 |
| **1556** | valerie agostino | 10 | 51.0 | 1 | 0.149 | 0.230 |
| **1557** | valerie agostino | 11 | 16.0 | 1 | 0.165 | 0.102 |
| **1558** | valerie agostino | 12 | 90.0 | 0 | 0.000 | 0.926 |

In [ ]:

```
freq_name = test_df_results.query("episode_col >= 8")["name"].unique()
sns.lineplot(x="episode_col", y="ibs", hue="name",
                data=test_df_results.query(f"name.isin({list(freq_name)})
```

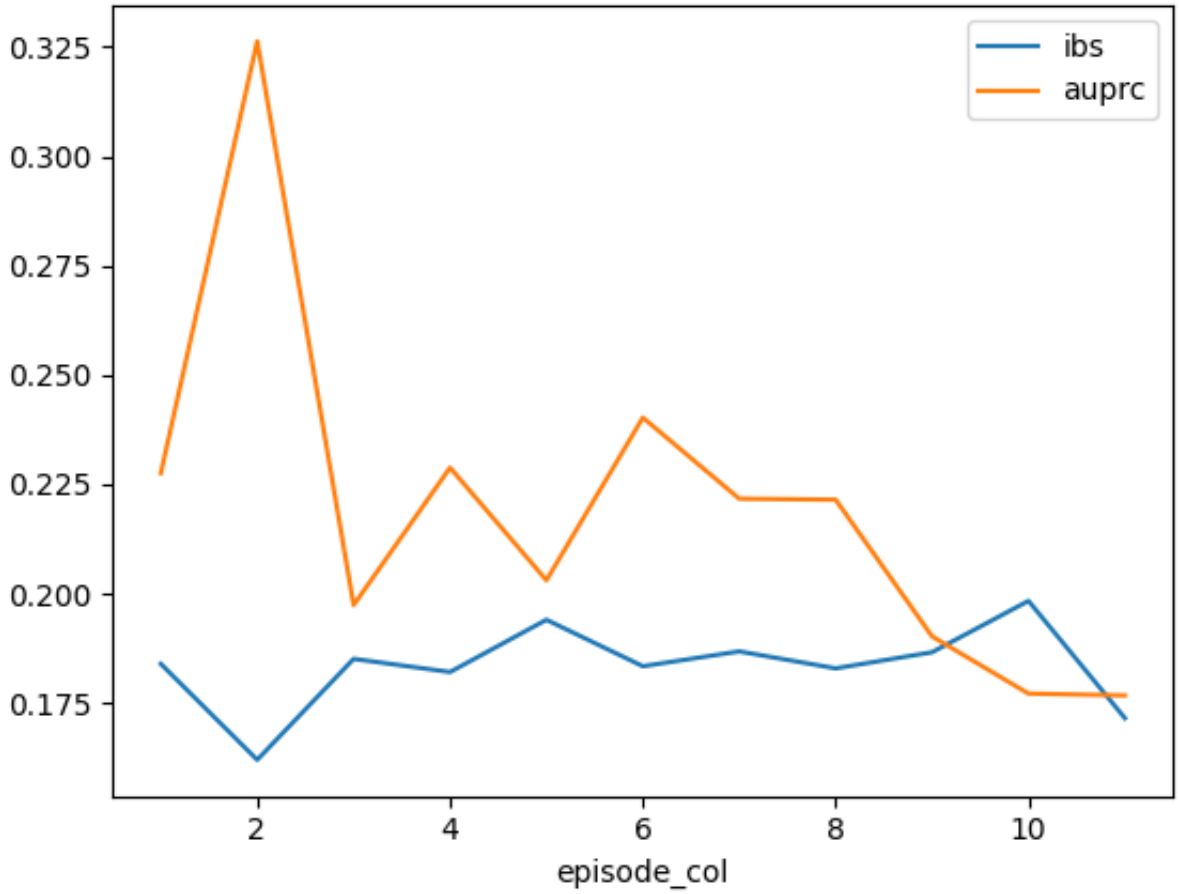
Out[ ]:

```
<Axes: xlabel='episode_col', ylabel='ibs'>
```




In [ ]:
```
test_df_results.query(f"name.isin({list(freq_name)}) and (event == 1)").g
```

Out[ ]:
```
<Axes: xlabel='episode_col'>
```



In [ ]:
```
# tr_pred = 1 - model.predict_survival(train_df, times).values.T
tr_pred = model.predict_cumulative_hazard(train_df, times)
tr_max = np.quantile(tr_pred.max(), 0.95)

# pred = 1 - model.predict_survival(test_df, times).values.T
# test_df.drop_duplicates(["name"], keep='first')
pred = model.predict_cumulative_hazard(test_df, times)
# pred[:] = train_df.groupby("name")["event"].sum().mean()  // baseline c
print("pred shape:", pred.shape)
print("len(test_df):", len(test_df))
metric = RecurrentCountError()

recurrent_error = metric.compute(
    survival_train=None,
    survival_test=test_df,
    estimate=pred / tr_max,
    times=times
)

mean_events = test_df.groupby("name")["event"].sum().mean()
print("Mean event =", mean_events)
print("Integrated cumulative event error:", recurrent_error)
```

```
pred shape: (200, 1623)
len(test_df): 1623
Mean event = 0.9207100591715977
Integrated cumulative event error: 0.2867342182519552
```

In [ ]:
```
pred
```

Out[ ]:

| | 0 | 1 | 2 | 3 | 4 | 5 | |
|---|---|---|---|---|---|---|---|
| **0.000000** | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000 |
| **7.537688** | 0.000513 | 0.003633 | 0.004160 | 0.002528 | 0.000618 | 0.001288 | 0.001 |
| **15.075377** | 0.001491 | 0.010560 | 0.012090 | 0.007349 | 0.001796 | 0.003743 | 0.003 |
| **22.613065** | 0.002788 | 0.019749 | 0.022612 | 0.013744 | 0.003358 | 0.007001 | 0.007 |
| **30.150754** | 0.003865 | 0.027374 | 0.031343 | 0.019051 | 0.004655 | 0.009704 | 0.009 |
| **...** | ... | ... | ... | ... | ... | ... | |
| **1469.849246** | 0.283391 | 2.007274 | 2.298296 | 1.396941 | 0.341349 | 0.711592 | 0.71 |
| **1477.386935** | 0.289903 | 2.053402 | 2.351113 | 1.429044 | 0.349194 | 0.727945 | 0.727 |
| **1484.924623** | 0.291038 | 2.061443 | 2.360319 | 1.434639 | 0.350561 | 0.730795 | 0.730 |
| **1492.462312** | 0.291038 | 2.061443 | 2.360319 | 1.434639 | 0.350561 | 0.730795 | 0.730 |
| **1500.000000** | 0.291828 | 2.067039 | 2.366727 | 1.438534 | 0.351513 | 0.732779 | 0.732 |

200 rows × 1623 columns

In [ ]:
```
test_df.drop_duplicates(["name"], keep='first')
```

Out[ ]:

| | name | episode_col | entry | dur | event | age | time_since_last_arrest | av |
|---|---|---|---|---|---|---|---|---|
| **0** | aaron hammond | 1 | 0.0 | 699.0 | 0 | 25 | 688.0 | |
| **1** | aaron villacampa | 1 | 0.0 | 107.0 | 0 | 26 | 106.0 | |
| **2** | abraham ballestas | 1 | 0.0 | 35.0 | 0 | 52 | 34.0 | |
| **3** | acie canteen | 1 | 0.0 | 215.0 | 0 | 27 | 214.0 | |
| **4** | adam gillstrap | 1 | 0.0 | 630.0 | 0 | 35 | 628.0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **1618** | zachary payton | 1 | 0.0 | 34.0 | 0 | 28 | 33.0 | |
| **1619** | zachary whalen | 1 | 0.0 | 634.0 | 0 | 24 | 632.0 | |
| **1620** | zaheer khan | 1 | 0.0 | 184.0 | 0 | 26 | 183.0 | |
| **1621** | zhivago walker | 1 | 0.0 | 126.0 | 0 | 30 | 125.0 | |
| **1622** | zuberi floyd | 1 | 0.0 | 757.0 | 0 | 22 | 756.0 | |

845 rows × 9 columns